

**Title: Multi-User LAN Communication System**

**Course:** Computer Networks

**Instructor:** Dr. Sanjeet Kumar Nayak

**Team members:**

K Shashank Reddy – CS23I1060

E Umesh Chandra – CS23I1036

## 1. Introduction

The **Multi-User LAN Communication System** is a standalone, server-based collaboration platform that enables real-time communication over a Local Area Network (LAN). It integrates **video conferencing, audio conferencing, screen sharing, text chat, and file sharing** into a single cohesive application.

The system is designed to operate in **offline or restricted network environments** where internet connectivity is unavailable. It serves as an all-in-one communication suite for internal networks such as offices, campuses, and laboratories.

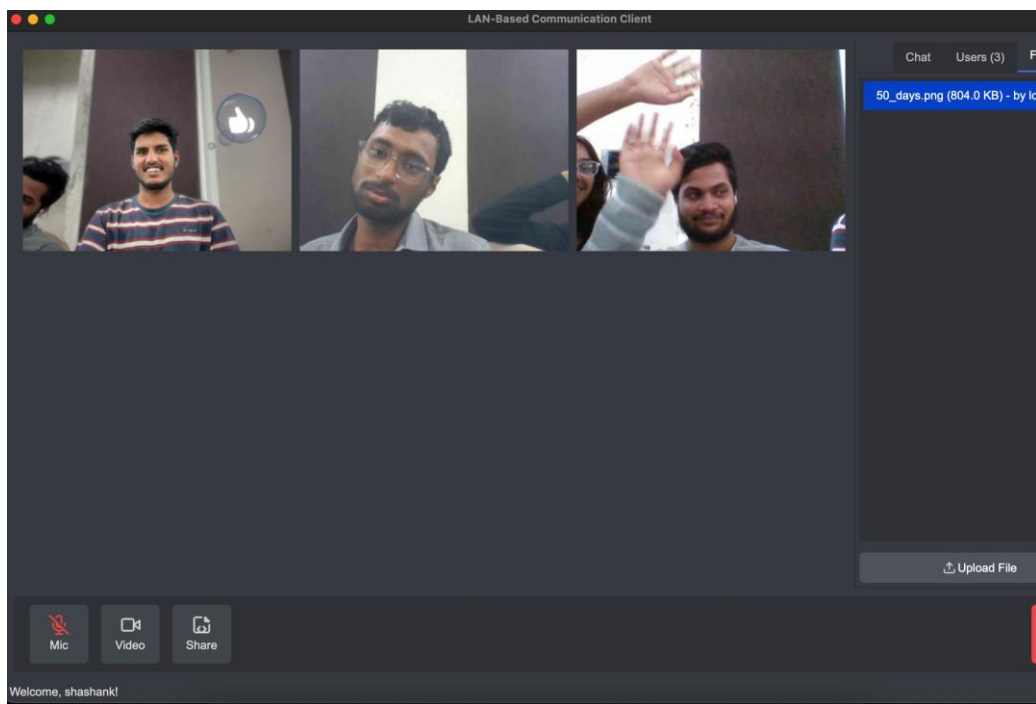
## 2. Objectives

- To design and implement a **robust LAN-based communication system** using socket programming.
- To enable **real-time interaction** through video, audio, and chat without requiring internet access.
- To provide **modular communication features** including file transfer and screen sharing.
- To ensure **low latency** and **efficient bandwidth utilization** for smooth communication.
- To develop a **user-friendly GUI** integrating all functionalities in one interface.

## 3. Core Functional Modules

### 3.1 Multi-User Video Conferencing

- **Client-Side:** Captures video from webcam, compresses, and sends via UDP.
- **Server-Side:** Receives streams from all clients, rebroadcasts to others.
- **Display:** Each client shows all participants in a grid layout.



## 4. Communication Protocols

| Module                  | Protocol | Description                                                   |
|-------------------------|----------|---------------------------------------------------------------|
| Video Conferencing      | UDP      | Low-latency transmission of compressed video frames.          |
| Audio Conferencing      | UDP      | Fast audio packet transmission for real-time playback.        |
| Screen/Slide Sharing    | TCP      | Reliable transfer of high-quality images/screens.             |
| Text Chat               | TCP      | Reliable message delivery and broadcasting.                   |
| Video and Audio Sharing | TCP      | Guaranteed, error-free file transfer and download management. |

## 5. Core Functional Modules

### 5.1 Multi-User Video Conferencing

- **Client-Side:** Captures video from webcam, compresses, and sends via UDP.
- **Server-Side:** Receives streams from all clients, rebroadcasts to others.
  - **Display:** Each client shows all participants in a grid layout.

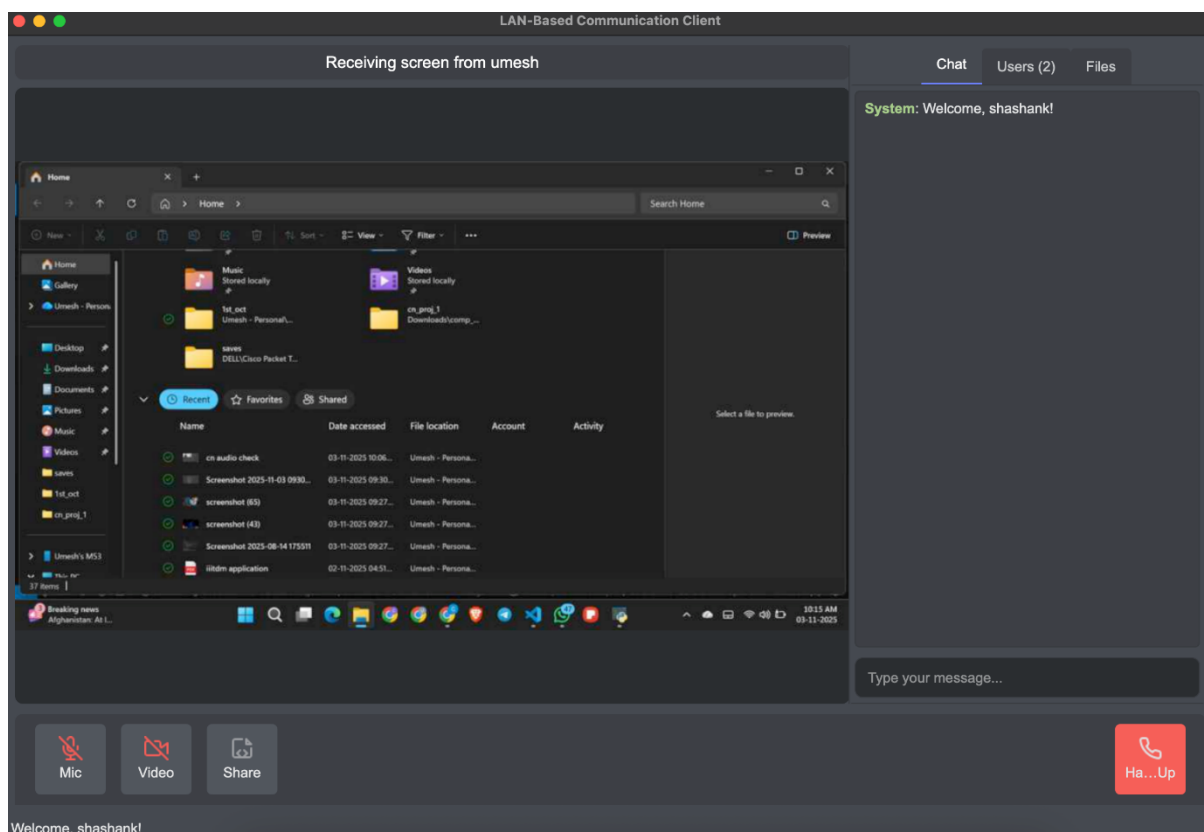
### 5.2 Multi-User Audio Conferencing

- **Client-Side:** Captures microphone input, encodes, and transmits.
- **Server-Side:** Mixes all audio streams into one and broadcasts to all clients.
- **Client Playback:** Decodes and plays through speakers/headphones.

Link for video-audio call testing : - [Drive Link](#)

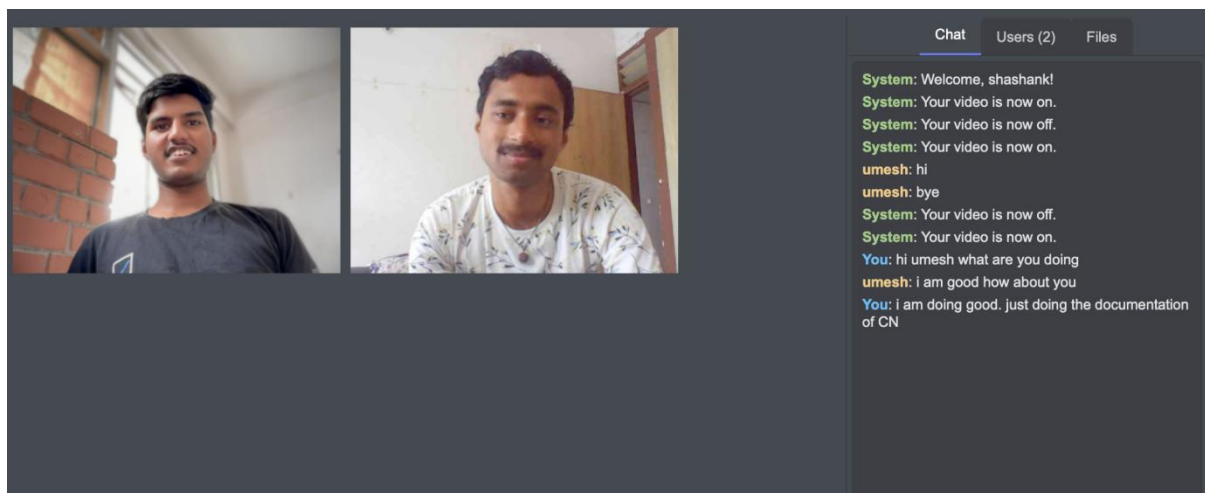
## 5.3 Slide & Screen Sharing

- **Presenter Mode:** One user acts as a presenter at a time.
- **Transmission:** Presenter's screen captured as compressed frames (TCP).
- **Broadcast:** Server sends frames to all participants for synchronized viewing.



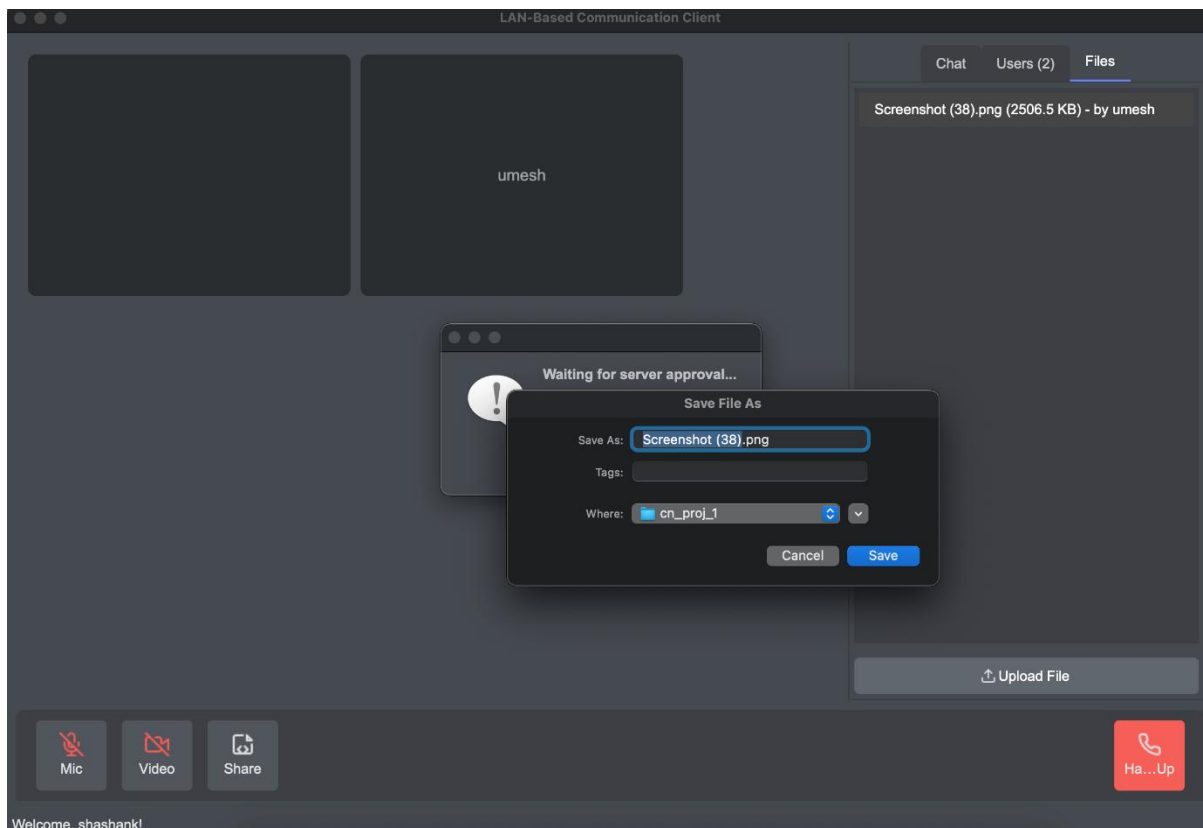
## 5.4 Group Text Chat

- **Transmission:** Messages sent via TCP to the server.
- **Broadcast:** Server redistributes messages to all connected clients.
- **Display:** Chat window maintains message history with sender names and timestamps.



## 5.5 File Sharing

- **Upload:** User selects a file and uploads via TCP to the server.
- **Distribution:** Server announces file availability to all clients.
- **Download:** Clients can request files; progress shown in the UI.



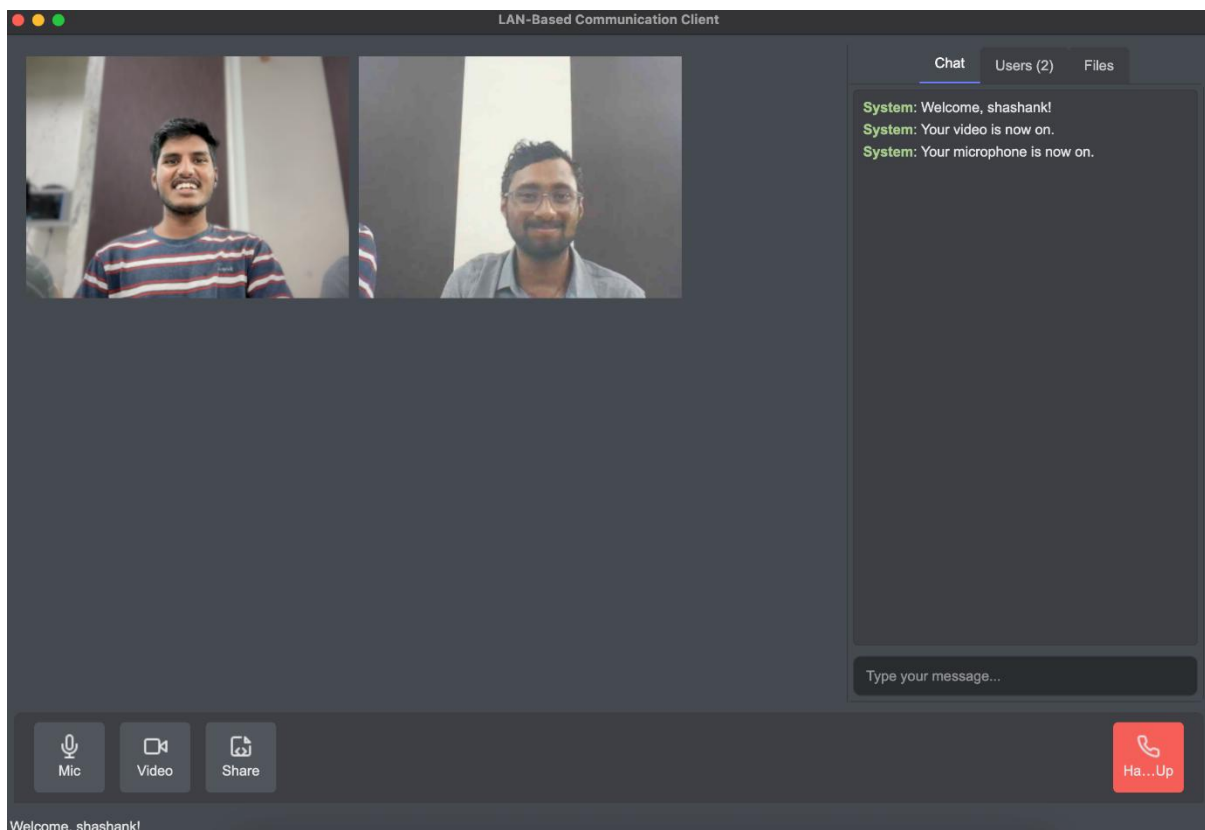
## 6. Technologies Used

| Component            | Technology / Library                                     |
|----------------------|----------------------------------------------------------|
| Programming Language | Python / C++ / Java (as per implementation)              |
| Networking           | TCP/IP, UDP Sockets                                      |
| GUI                  | Tkinter / PyQt / Kivy                                    |
| Multimedia           | OpenCV (video), PyAudio / PortAudio (audio), PIL (image) |
| Threading            | Multithreading for handling concurrent clients           |
| Compression          | zlib / OpenCV Video Codec for video frames               |

## 7. User Interface Overview

The GUI provides an integrated workspace containing:

- **Video Panel:** Displays all participant streams.
- **Audio Controls:** Mute/unmute microphone and speakers.
- **Chat Section:** Group text conversation pane.
- **Screen Sharing Area:** Display of presenter's screen.
- **File Panel:** Upload, download, and view available files.



The interface design focuses on simplicity and ease of use.



## 8. System Flow

### 8.1 Server Workflow

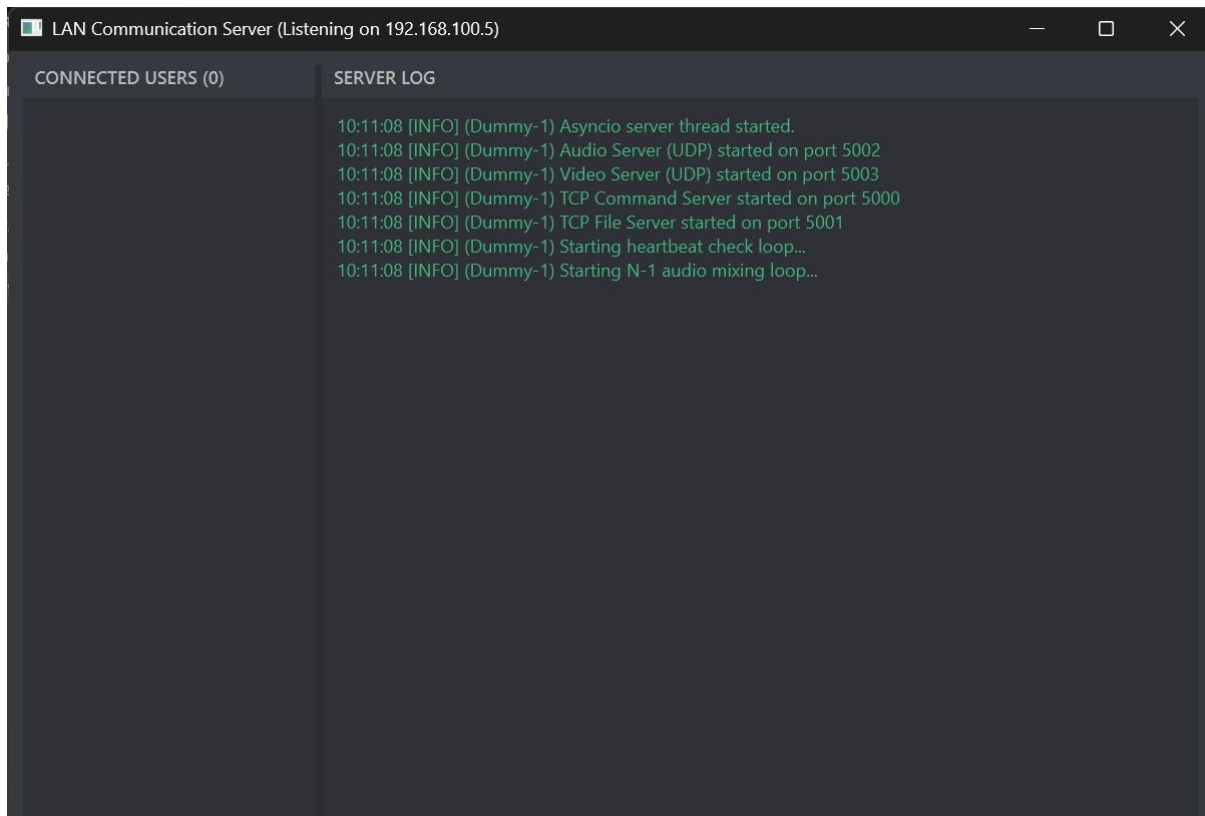
1. Initialize server sockets (TCP & UDP).
2. Accept multiple client connections.
3. Manage authentication (if implemented).
4. Handle message routing, stream mixing, and broadcasting.
5. Maintain session list and disconnect idle users gracefully.

### 8.2 Client Workflow

1. Connect to the server via LAN IP.
2. Initialize audio, video, and chat modules.
3. Send/receive media streams and messages concurrently.
4. Handle user interface interactions.
5. Gracefully disconnect when leaving the session.

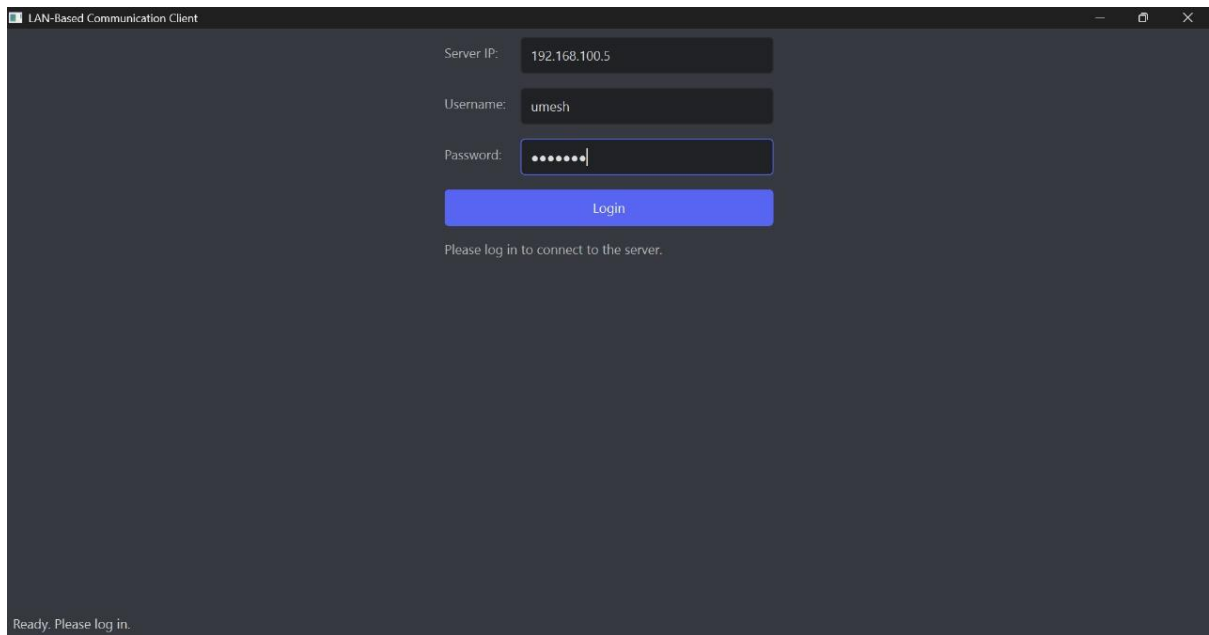
## 9. Performance and Optimization

- **Low Latency:** UDP used for audio/video streams to minimize delays.
- **Thread Management:** Separate threads handle each communication channel.
- **Compression:** Real-time encoding reduces bandwidth consumption.
- **Error Handling:** Retry and timeout mechanisms implemented for TCP modules.
- **Scalability:** Supports multiple concurrent users with efficient resource allocation.



## 10. Security Considerations

- Internal LAN access only (reduces external attack surface).
- Basic authentication for client connections (optional).
- Data validation on server to prevent malformed packets.
- Future enhancement: implement AES encryption for chat and file transfers.



## 11. User Guide

### 11.1 Server Setup

1. Launch the server application on a machine connected to the LAN.
2. Note the server's local IP address (e.g., 192.158.1.10).
3. Ensure the required ports (TCP/UDP) are open.

### 11.2 Client Setup

1. Launch the client application.
2. Enter the server's IP address and connect.
3. Once connected, the user can:
  - a. Join a video/audio conference.
  - b. Participate in group chat.
  - c. Share screen/slides or upload files.

## 11.3 Termination

- Clients can disconnect manually.
- The server logs user disconnections automatically.